

-Pitch

In the modern day, the complexity of software that allows users to communicate is deeply impressive. Social Media and other networking applications allow us to communicate with a wide array of people, essentially in real-time. On top of that, it's easy. There are no major hoops to jump through to be able to do it. In my opinion, it's so easy that it almost becomes unconscious. As someone that values both communication and wants to understand the underpinnings of it, I think the best way is to take a step back.

In order to deeply understand most topics, I like to break it down into increasing layers of abstraction. When I began to consider topics concerning the topics of communication, I immediately wanted to make something that had a 'lower-level' feel than our normal social media networks. In fact, what immediately came to mind was the UNIX 'talk' command that allowed two users on separate terminals but the same computer to talk to each other. While I believe this topic to be interesting and ultimately part of the pipeline in creating our modern day social media, everyone is too used to personal computers to be able to directly relate to this.

After further consideration, I remembered Internet Relay Chat servers. While still in limited use today, they were widely used, especially by technical individuals, before social media came around. These systems were relatively simple chatrooms that were hosted on servers. They allowed multiple users to connect at once and discuss. They had simple functions and also allowed a great deal of customization per server. When exploring how internet communication systems work while still being able to 'tinker', the systems surrounding IRC servers are a great way to do so.

From that conclusion I decided to attempt to write an IRC-like application myself. My intention from the start was not to create another social media platform that is endlessly modern (and a little dull, if I'm honest), but I still wanted to make it accessible for computer science students, like myself, to have relatively easy access to it. The method I decided on was to write these programs with no external libraries (outside of those that come default with the GNU C Compiler).

The intent of this project is not to solve some communications gap that is there for the normal user that is not interested in anything other than ease of communication. Instead, it is to abstract what communication over the internet actually takes for the Computer Science students that are interested in it. I honestly think that we stepped back to where university students were the ones developing their own means of digital communication, even in today's day and age, it would make better Computer Scientists out of us all.

With that intent, I made 'TCP-CHAT'. It is a Command Line Interface based chat room that is somewhat derivative of IRC, albeit with less features. It allows for multi-user interactions as well as private messaging. It is written in C while only using the GNU C Library. It can easily be expanded to be a much better application than it is right now, and I have many things that I would do differently after learning the basics of C socket programming. However, that is also kind of the point. I wrote TCP-CHAT as an excuse to 'put my money where my mouth is', when it comes to socket programming and I can say that I learned a lot from it.

-Technical Explanation

TCP-CHAT has a relatively simplistic design, albeit with a relatively complicated implementation, partially because of my inexperience with handling strings in C, and partially because there are a significant amount of function calls to glibc functions.

At a high level, the server program starts off by establishing an endpoint called a socket that listens for other connections. Once that is ready, it can begin to add connections as they come in while still being

able to do other logic. It does this by using the `poll(fds,nfds,100);`. This function essentially allows processes to continue to happen in the loop without being caught if nothing is happening.

```
active = poll(fds,nfds,100);
/* need to hand reoccupying disconnected connections */
if(clientsConnected < (sizeof(clients) / sizeof(clients[0]))) {
    if(fds[0].revents & POLLIN) {
        /* refresh socklen or accept() starts getting weird */
        socklen = sizeof(address);
        if((clients[clientsConnected] = accept(inputSock, (struct sockaddr*)&address, &socklen)) < 0) {
            perror("accept");
            continue;
        }
    }
}
```

In this photo, it'll only add a new client socket if there is one waiting in a queue that a `listen()` function sets up. This allows other logic to happen in this same loop without having to wait on the `accept()` function, like you normally would. When using `poll`, I'm also able to add a delay to it that acts as a `psuedo sleep()` function so that it is not looping unnecessary amounts. For the program I found 100ms to be acceptable.

I'm using fixed size arrays for most things in this program, which is one thing that I would correct if I was to do it all over again, instead using dynamic arrays or a map. The fixed size arrays work, but since everything is set up essentially as parallel arrays it becomes difficult when clients disconnect. This issue is primarily because I did not plan ahead on using file descriptors. Another option would be multithreading.

Once a client socket is in the `clients` array, `poll[]` and the file descriptor of `clients[]` are used to check if there is any information available from any of the client sockets. If there is, the input is read and parsed based on the various commands available. The output of the parsing function is fed into a switch statement that executes various command functions based on the client input.

If a client disconnects, it's socket is closed and the data stored in those fixed-length arrays is removed. However, the `clientCount` does not drop down because the `poll()` function is reliant on it. This is probably the biggest issue in the program and it is entirely because I did not anticipate using `poll()`.

On the client side, `poll` and the file descriptors are also used to update and send things to `stdout` while not having to wait for `stdin`. It is much more simplistic as I chose to leave all command parsing to the server side, including the disconnect command.

The one major limitation that I had with only using `glibc` was not having a great way to write a good cursor/input buffer without having to rewrite a significant amount of how inputs work. It is possible by setting the `stdin` buffer to output every keystroke, but that becomes quite complicated.